Reducing Bank-Synchronization Cycles of a Zero-Skipping FPGA CNN Core by Offline Channel Permutation

(Author names)
(Affiliation)

(e-mail)

Abstract—A zero-skipping convolution core whose input banks run in lockstep waits, in every window, for the bank with the most nonzero activations. In our integer CNN core for a Zynq XC7Z020 FPGA, the bank of a tap is its index mod 4 in a (ky, kx, channel) window, so where the channel count is a multiple of 4 the channel order decides which channels share a bank. We fix this order offline on calibration images and fold it into the adjacent layers, leaving the bitstream and the integer output unchanged; a cycle model shows from the layer shapes alone where this cannot pay. At four values per cycle, two relabelings of a 32-channel denoiser cut the core cycles measured on one image by 7.02% and 7.60% (transcribed from run notes), as a model pushed before the export predicted to the cycle once reported DMA stalls were removed; under a protocol pushed before the fits, the pooled count fell on all ten networks studied. A data-free skewed tap layout, counted after the study and not run on the board, saves more: 9.20% pooled over 79 images against 7.46% and 8.10%, and more than both relabelings on every image. Whether a relabeling fitted on top of the skew adds to it is left to a new pre-registered test.

Keywords—zero skipping, activation sparsity, load balancing, channel permutation, FPGA, CNN accelerator, cycle model

I. Introduction

A core that skips zero activations saves multiplications [1], [2], but units that advance together make each window cost as much as the busiest one [1], [3]. Known remedies for activation imbalance change the network or add hardware: S2TA and a patent bound the nonzeros per block or bank by discarding activations [4], [5], and Sense and Crane rebalance at run time [6], [7]; Sense notes that input maps "cannot be trained offline to balance IFM workload" [6]. Offline, function-preserving reorderings exist, but they balance weights [8]–[10].

In our core four tap banks run in lockstep, and in 32- and 64-channel layers all nine taps of a channel share the bank given by its index mod 4. A channel reordering folded into the adjacent layers [8], [11] moves nonzeros between banks with the function exact, but balances them only on average, as activations change with the input. We give a shape-only condition necessary for a layout to change a layer's cycles (Section II), a board measurement predicted to the cycle by a model pushed before the export (Section V-A), and a ten-network study under a protocol pushed before the fits, with baselines (Sections V-B, V-C), of which a data-free skew of the taps, counted after the study, beats both fitted relabelings.

II. Core, Cost Model and Bank Layout

Core. The host builds each layer's windows (im2col), sends them with the layer's weights and requantizes between layers; the core computes every convolution with INT8 weights, 7-bit unsigned activations and INT32 accumulation. Its 64 multipliers form P = 16 output lanes times T = 4 tap banks; its window port takes W values per cycle (W = 2 only in RTL simulation). In sparse mode, used throughout, each bank processes only its nonzero taps.

Cost of a window. For a window with K taps and COUT outputs, and m the largest number of nonzero taps in one bank, the core spends

$$C = \max(\lceil K/W \rceil + 1, \lceil COUT/P \rceil \cdot \max(1, m), COUT) \quad (1)$$

cycles: loading the next window, computing $\lceil COUT/P \rceil$ output groups, and emitting COUT results. Each layer adds a fill and drain of min(load, compute of the last window) + 3 + g cycles, g being the outputs in the last group of P, a rule read off board runs of four networks at W = 1 and 4; the first layer, which reads the image, has a measured constant of 17 cycles. We call (1) summed over the windows the count, and the count plus the fill and drain the exact prediction.

Bank of a tap. In (ky, kx, channel) order, tap $t = s \cdot N + p$ (s the 3×3 position, p the channel, N the input channels) sits in bank t mod T, which is p mod 4 when 4 divides N: each bank holds N/4 whole channels (Fig. 1a), and a permutation of the channels decides which share a bank (Fig. 1b), keeping the window order a hardware line buffer would produce. As this host builds the windows, the taps can also be reordered together with the

consumer's weight columns, without a bitstream change; the INT32 sums stay equal, as |partial sum| ≤ K·127·128 < 2^31. The data-free skew puts channel p of position s in slot s·N + (p + s) mod N, bank (p + s) mod 4, spreading each channel's nine taps 3/2/2/2 over the banks (Fig. 1c).

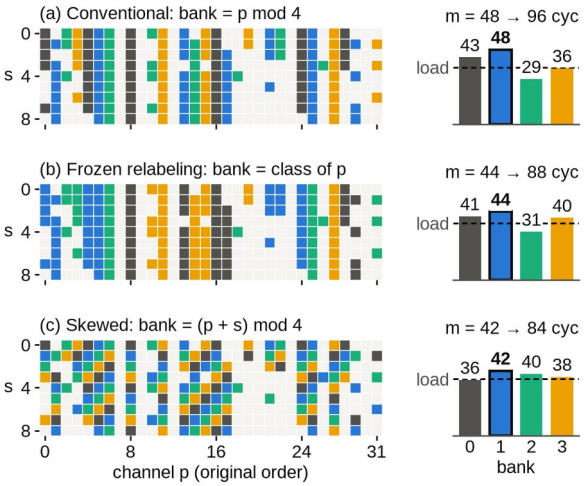


Fig. 1. A conv5 input window of the board network on Set12 05.png (of the windows compute-bound at W = 4, the first in raster order with the median m of 48). Rows: positions s; columns: channels p; filled: nonzero, coloured by bank. (a) Bank p mod 4; (b) Frozen (16 of 32 channels change bank); (c) skew, bank (p + s) mod 4. Bars: nonzeros per bank; the fullest costs 2m cycles; dashed: 2m = load (73).

When a layout can pay. A bank holds at most ⌈K/T⌉ taps, so if

$$\lceil \mathrm{COUT}/P \rceil \cdot \lceil K/T \rceil \le \max(\lceil K/W \rceil + 1, \mathrm{COUT}), \quad (2)$$

no window is compute-bound for any input, and no layout can change the count; failing (2) is necessary for a gain. Without the rounding, (2) reads COUT ≤ P·T/W, the ridge point of a roofline [12] whose bandwidth roof is the load. By Table I, 32-output hidden layers can pay only at W = 4, layers with 45 or 64 outputs also at W = 2, none at W = 1. As references, perfect balance, m = ⌈n/T⌉ for n nonzero taps, bounds every assignment of a window's taps to lockstep banks, and the floor is the count if no window were compute-bound.

TABLE I. WHERE A LAYOUT CAN CHANGE THE COUNT, FROM (2). MAX.: ⌈COUT/P⌉·⌈K/T⌉. FLOOR: MAX(⌈K/W⌉ + 1, COUT). NO LAST LAYER (COUT ≤ 4) CAN PAY.

Hidden layers	K / COUT	Max.	Floor, W = 2, 4	Pays, W = 1, 2, 4
A, B-32 ReLU	288 / 32	144	145, 73	no, no, yes
B	576 / 64	576	289, 145	no, yes, yes
maxout, avg.	288 / 64	288	145, 73	no, yes, yes
10 × 45	405 / 45	306	204, 103	no, yes, yes

III. FUNCTION-PRESERVING RELABELING

Folding. Let π map a position p of a hidden map to its original channel. The producer computes its output channels in the order π, with weight rows, biases and requantization scales reordered, and the consumer's weight column for tap s·N + p comes from channel π(p). As ReLU and requantization act per channel and the consumer adds the same INT32 products, the integer output is identical for every input. We permute the outputs of conv1 to conv(L−2) of an L-layer network; the last layer, which can never pay, keeps its input. In the maxout [13] and averaging networks, which keep the maximum or mean of maps j and j + N of 2N computed maps, both maps move with π. For N = 45, tap s·45 + p is in bank (s + p) mod 4: the conventional layout is already the skew.

Fitting. Relabeled windows are original windows with permuted columns, so each layer is fitted alone. The recipe, Frozen, draws 2048 windows per fitted layer from Set12 01.png (Cal-1; σ = 25, one noise draw), minimizes their mean of (1) at W = 4, and searches from the identity with 400 proposals, each swapping two channels of different classes and kept only if the objective falls (7.1–28.8 s per network on a 4-core x86 server). A second layout of the board network, Multi-start, adds Set12 02–03, two noise draws per image (Cal-3) and four random starts (2005 objective evaluations per layer against 401); it was chosen for the board after its BSD68 counts had been seen.

IV. EXPERIMENTAL SETUP

Networks and images. Ten trained integer DnCNN-style networks [14] of six architectures, for Gaussian noise of σ = 25 on 7-bit inputs: A, 7 layers of 32 channels; B, 10 layers of 64 channels behind a 2×2 space-to-depth input; B-32, the same with 32 channels, with ReLU (training seeds s1–s3), maxout over pairs of 64 computed outputs (s1–s3) or their mean (s2); and a 45-channel ReLU variant. All but A had quantization-aware training [15]. The board network, B-32 ReLU s1, reaches 29.54 dB on Set12 05.png; 37–55% of its hidden layers' window values are nonzero. Test images: Set12 [14] and BSD68 [16], 80 images, one noise draw each; savings exclude the calibration images unless stated.

Board. The W = 4 bitstream for the XC7Z020 (xc7z020clg484-1) closes timing at 100 MHz (worst negative slack +0.130 ns), uses 11,151 LUTs (21%), 71 of 140 BRAM36-equivalent tiles and no DSP slice, and ran at the board preset's 50 MHz. One bitstream (core ID 0x4d410303) ran all three layouts; per layer the driver checks the output against the integer model and reads the cycle counter CYCLES and the DMA-wait counter IN_STALL. On 05.png the Original ran once as a canary, then each layout three times in alternation. These board cycles are transcribed from run notes, each checked by a script against its cited line (an earlier run of the Original is a committed raw report). All other cycle numbers are counts over the windows of the host emulation, which reproduces every network's reference PSNR on all 80 images.

What was fixed when (GitHub push times, UTC). The W = 4 count model preceded the bitstream (7e81f6b, 10-17 07:52);

the fill-and-drain rule (6dc0e60, 13:15; b37f971, 15:49) was read off board runs of the four unpermuted networks, so the board network's Original layers are in sample. Frozen's per-layer count savings on 05.png (9cf9c79, 18:38) and both exports (c2ad6ad, 19:25; 45b4f16, 21:00) preceded the board runs (10-18). A protocol (c2299e2, 10-18 18:32) then fixed the recipe, the evaluation, condition (2) per layer and four hypotheses before any fit on the other nine networks, and the fits and evaluation code were pushed (b3461cf, 19:29) before the results were written. The skew was counted afterwards (2928d28, 10-19), outside that protocol.

V. Results

A. Board Measurement

On Set12 05.png (Table II), Multi-start lowered the board total from 13,289,030 to 12,278,419 cycles (−7.60%) and Frozen to 12,355,814 (−7.02%). The repeats gave the same numbers, except 12 more cycles in Frozen's third run, all in conv10, where IN_STALL was also 12; none of their 90 layer runs had an output mismatch, and the notes report the denoised images as byte-identical. In all 63 comparisons of layers, sums and totals (canary, repeats and the earlier run), CYCLES − IN_STALL equaled the exact prediction; as the repeats are deterministic and conv1 and conv10 see the same windows in every layout, nine of them are independent and out of sample, all on one image: Multi-start's conv2–conv9 and Frozen's conv2–9 sum.

TABLE II. CORE CYCLES OF B-32 RELU S1 ON SET12 05.PNG AT W = 4. UPRIGHT: BOARD CYCLES TRANSCRIBED FROM THE RUN NOTES, AND SAVINGS FROM THEM. ITALIC: EXACT PREDICTION OR COUNT. M: MULTI-START; F: FROZEN.

Layer	Original	Multi-start	Frozen	Saving, M/F (%)
conv1	524,305	524,305	*524,305*	0.00/*0.00*
conv2	1,504,752	1,378,763	*1,391,312*	8.37/*7.54*
conv3	1,435,005	1,344,509	*1,362,489*	6.31/*5.05*
conv4	1,448,350	1,311,511	*1,311,569*	9.45/*9.44*
conv5	1,562,936	1,371,900	*1,387,589*	12.22/*11.22*
conv6	1,369,221	1,272,788	*1,275,254*	7.04/*6.86*
conv7	1,375,441	1,281,427	*1,291,330*	6.84/*6.12*
conv8	1,289,021	1,207,219	*1,212,745*	6.35/*5.92*
conv9	1,436,476	1,242,474	*1,255,698*	13.51/*12.58*
conv10	1,343,523	1,343,523	*1,343,523*	0.00/*0.00*
conv2–9	11,421,202	10,410,591	10,487,986	8.85/8.17
Total	**13,289,030**	**12,278,419**	**12,355,814**	**7.60/7.02**
Count	*13,288,456*	*12,277,883*	*12,355,274*	*7.60/7.02*
Fill/drain	*574*	*536*	*540*	—

Notes to Table II. Per layer: repeat 1; Frozen was reported on the board only as totals and conv2–9 sums. In every upright cycle cell, board − IN_STALL − exact prediction = 0 (per-layer IN_STALL 0: by the notes, only Frozen's third repeat stalled).

Mechanism (counted). The fullest bank's mean nonzero count fell in every relabeled layer (conv9, Multi-start: 43.04 to 35.10), and more windows reached the load floor of 73 cycles (conv9: 27.8% to 61.2%); in the window of Fig. 1, Frozen lowers m from 48 to 44. Perfect balance would give 11,761,449 cycles (−11.49%), the floor 11,436,032 (−13.94%); Multi-start removed 66.2% of the perfect-balance saving, Frozen 61.1%.

B. Study of Ten Networks

The protocol fixed two checks and two sign predictions. H0: our code reproduced the earlier Frozen layout in all eight layers, and folding it reproduced the Frozen export byte for byte. H1: wherever (2) holds, at W = 1 on all networks and at W = 2 on A and B-32 ReLU s1–s3, the relabeled count equaled the original on all 80 images. H2: the pooled W = 4 saving over the 79 images was positive on all ten networks, 1.09–10.80% (Fig. 2; the sign was known for B, maxout s1 and the board network). H3: the pooled W = 2 saving was positive on all six networks where (2) allows it, 3.36–7.37% on five and 0.022% on the 45-channel network, where 16 of 632 image-layers got worse (at most 3 on any other network) and conv7 rose by 0.0001%, the only pooled increase of any layer. Only signs were predicted. No image counted more cycles on any network at any W.

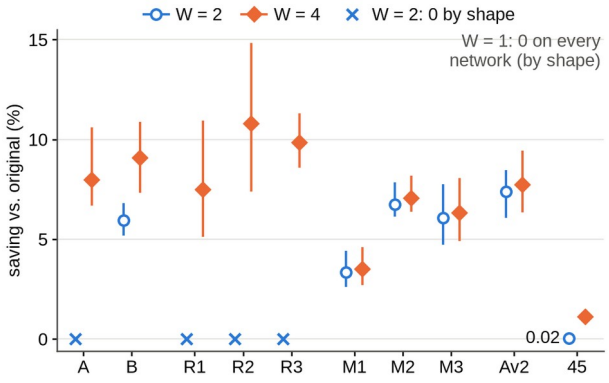


Fig. 2. Pooled saving of the recipe over the 79 images other than the calibration image (count, conv1–convL); lines: per-image range; ×: (2) holds in every layer, saving exactly 0. A: 7 × 32; B: 10 × 64; R1–R3, M1–M3: B-32 ReLU and maxout, seeds s1–s3 (R1: board network); Av2: averaging, s2; 45: 10 × 45 ReLU.

Maxout s1 gained only 3.50% at W = 4; without per-network headroom, a weak fit cannot be told from a small opportunity. Maxout and averaging save almost as much at W = 2 as at W = 4: with four output groups, their compute exceeds the W = 2 load of 145 cycles once the fullest bank holds more than 36 nonzeros. In emulation every folded network reproduced the original's PNG bytes on Set12 05.png and BSD68 test001.png. As the protocol's search routine rejects N = 45, that network was fitted with a general bank rule (identical layouts on the 69 layers where both apply).

C. Eighty Images and Baselines

At W = 4 (Fig. 3), Frozen saved 5.13–10.94% per image on the 79 images other than its calibration image (median 7.22%, pooled 7.46%), Multi-start 5.62–11.19% on the 77 other than its three (median 7.89%, pooled 8.11%); perfect balance would

save 7.99–16.62% (pooled 12.02%), the floor 15.01%. One of Frozen's 632 fitted image-layers counted more (BSD68 test028 conv3, +1.56%), none of Multi-start's 616. From the smallest saving, 05.png ranks 34th (Frozen) and 32nd (Multi-start) of 80, and Multi-start beat Frozen on 74 of the 77 images used by neither fit.

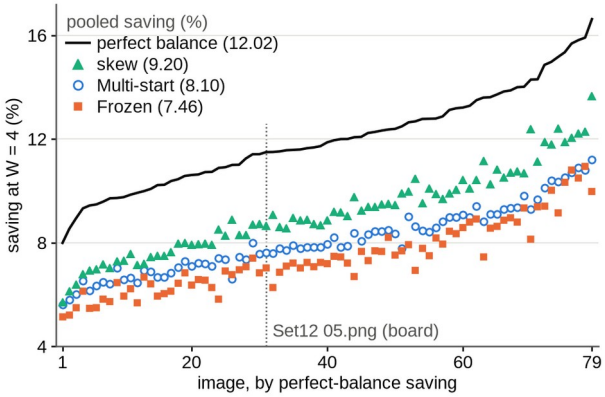


Fig. 3. Per-image W = 4 saving of the board network (count, conv1–10) on the 79 images other than Set12 01.png, sorted by the perfect-balance bound; legend: pooled saving. Set12 02–03 are Multi-start calibration images. Counts, not board runs.

TABLE III. LAYOUTS OF B-32 RELU S1 AT W = 4 (COUNT). CAL-1, CAL-3: SECTION III. P.I.: USES THE TEST IMAGE. POST HOC: COUNTED AFTER THE STUDY. RECIPE: FROZEN'S. FIVE-SEED ROWS: MEAN POOLED SAVING (RANGE OF THE SEEDS) AND PER-IMAGE RANGE OVER ALL SEEDS; 5×: 2005 EVALUATIONS PER LAYER. SHARE: (ORIGINAL − LAYOUT)/(ORIGINAL − PERFECT BALANCE), CYCLES SUMMED OVER THE IMAGES. RANDOM: RANGE AND SHARE FROM THE PER-IMAGE MEDIAN.

Layout (79 images; † 77)	Pooled (%)	Per image (%)	Share (%)
Random, median of 20	−2.31	−5.30 to 1.18	−19.2
Density, round-robin, Cal-1	3.59	1.23–6.52	29.9
Density, smallest total, Cal-1	3.69	1.76–5.64	30.7
Sense-like, smallest total (p.i.)	4.89	1.53–9.94	40.7
Frozen (board layout), Cal-1	7.46	5.13–10.94	62.1
Recipe, Cal-1 (7.46–7.78)	7.62	5.13–11.30	63.4
Recipe per test image (p.i.)	8.45	5.77–12.79	70.3
Multi-start, Cal-3† (8.11–8.26)	8.17	5.52–11.55	67.9
Greedy 5×, Cal-3† (8.07–8.36)	8.24	5.39–11.69	68.5
Skew, data-free (post hoc)	9.20	5.70–13.66	76.6

Table III compares other layouts. Twenty random layouts with eight channels per class saved −5.43% to +1.73% pooled (median −2.31%), so savings against the identity understate those against a random order. Six layouts after SparTen's density sort [8], using activation density (channels dealt by mean nonzero count to the classes in round-robin or snake order, or each to the class with the smallest total), captured 21–33% of the perfect-balance saving, the deployable fitted layouts 62–68.5%; a Sense-like sort [6] by each channel's nonzero count in the test image reached at most 58% of the recipe fitted on that image. This compares procedures and does not isolate co-activation; the count also depends on correlation among a channel's nine taps, which no relabeling can change. Changing one thing at a time, Multi-start's 0.63-point lead over Frozen on the 77 images splits into Frozen's seed (0.15; the lowest of five), five times the budget (0.29) and Multi-start's data (0.32), less its restarts (0.07) and seed (0.06); with equal data and budget, plain greedy search beat Multi-start on four of five seeds.

Skew (counted after the study). Applied to the inputs of conv2–conv9, the skew saved 9.20% pooled over the 79 images (median 9.02%, 5.70–13.66% per image), 76.6% of the perfect-balance saving, and fewer cycles than both relabelings on every image (Fig. 3), although the 79 include Multi-start's calibration images (on the other 77: 9.22% against 8.11%). On 05.png it counted 12,139,748 cycles (−8.64%, against 7.02% and 7.60% for the relabelings), and there it also beat the recipe on A, B and maxout s1 (10.01%, 10.91%, 5.61% against 7.67%, 9.36%, 3.47%). In the integer oracle, on 4096 windows per layer of 05.png, the permuted taps and weight columns left every accumulator unchanged. The relabeling is thus not the best static layout of this core.

VI. RELATED WORK

Balanced pruning [17], [18] and BRIM's offline weight reshaping from profiled activation statistics, with hardware for the residual imbalance [19], also change the function. SparTen sorts filters by density offline and reorders the next layer's weights [8], the closest precedent, with a weight objective; Pool and Yu fold a channel permutation for N:M pruning into the preceding layer [11]; Xu et al. and SparHiXcel-v2 order channels on FPGAs for weight sparsity [9], [10]; VARA permutes inputs offline to cluster co-zero activations, through an index unit [20]. Skewed storage gives conflict-free access to the rows and columns of a matrix in parallel memories [21]; our skew applies it to window taps. Channel permutation, its folding, activation profiling, greedy search and skewing are thus not new; our search (Crossref, arXiv, web; October 2026) found no work that spreads co-active channels over lockstep banks by an exact, folded relabeling. The contribution is empirical: how much of the bank-imbalance cost static layouts remove on unseen images, and cycle-exact prediction.

VII. LIMITATIONS

• **Tap-level layouts.** Committed: the skew's count on the board network (80 images) and four other networks (05.png only), and its exactness in the integer oracle. Open: the export and driver path for skewed windows and weight columns, its host time, a board run, the skew on all ten networks, and whether a relabeling fitted on top of the skew adds to its saving, under a new pre-registration separate from H0–H3.

• **Board evidence.** One network and one image were measured, and the fill-and-drain constants come from runs of the same image. Table II is transcribed from run notes that give neither Frozen's per-layer cycles nor image digests; the raw reports are not yet committed.

• **End-to-end effect.** Only core cycles are reported; the JTAG harness dominates (the earlier run of the Original spent 315.0 s in debugger sessions against 0.266 s of core time, so Multi-

start's 20.2 ms are 0.006%). Host im2col and requantization time, energy, and FIFO-decoupled banks, which could beat the per-window perfect-balance bound, were not measured.

• **Statistics and selection.** Only $\sigma = 25$ with one noise draw per image; W = 2 results are counts; calibration-image spread and confidence intervals were not measured. BSD68 was part of the networks' development, Multi-start was chosen after its BSD68 counts were seen, and the skew was counted after the study.

• **Stall counter.** This bitstream's IN_STALL misses DMA starvation at window boundaries (fixed in later RTL); the exact match excludes hidden stalls that differ between layouts, and the fill-and-drain rule's hold in RTL simulation, which has no DMA, suggests the constants contain none.

VIII. Conclusion

On a zero-skipping core with lockstep banks, the layer shapes tell where the tap-to-bank layout can matter, and a folded relabeling changed the measured cycles as counted, with identical output. It lowered the pooled count on all ten networks, but a data-free skew saved more on the board network; what a fitted layout adds to the skew, and how well layer shapes predict its gain, remain open. Scripts, layouts, per-image counts and the source of every number are in bankperm/ of https://github.com/qewr1234/FPGA.

Acknowledgment

(Acknowledgment)

References

[1] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. Enright Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 1–13, doi: 10.1109/ISCA.2016.11.

[2] A. Aimar et al., "NullHop: A flexible convolutional neural network accelerator based on sparse representations of feature maps," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 3, pp. 644–656, 2019, doi: 10.1109/TNNLS.2018.2852335.

[3] A. Parashar et al., "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proc. 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2017, pp. 27–40, doi: 10.1145/3079856.3080254.

[4] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2022, pp. 573–586, doi: 10.1109/HPCA53966.2022.00049.

[5] E. Yan, D. Xu, and J. Liu, "Bank-balanced-sparse activation feature maps for neural network models," U.S. Patent 12 236 341 B2, Feb. 25, 2025.

[6] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen, and Y. Kang, "Sense: Model-hardware codesign for accelerating sparse CNNs on systolic arrays," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 4, pp. 470–483, 2023, doi: 10.1109/TVLSI.2023.3241933.

[7] Y. Guan et al., "Crane: Mitigating accelerator under-utilization caused by sparsity irregularities in CNNs," *IEEE Trans. Comput.*, vol. 69, no. 7, pp. 931–943, 2020, doi: 10.1109/TC.2020.2981080.

[8] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2019, pp. 151–165, doi: 10.1145/3352460.3358291.

[9] J. Xu, H. Pu, and D. Wang, "Sparse convolution FPGA accelerator based on multi-bank hash selection," *Micromachines*, vol. 16, no. 1, Art. no. 22, 2025, doi: 10.3390/mi16010022.

[10] A. Zarei and S. Vakili, "A flexible sparsity-aware FPGA accelerator with column-wise compression for efficient CNN inference," arXiv:2607.19248, 2026.

[11] J. Pool and C. Yu, "Channel permutations for N:M sparsity," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 13316–13327.

[12] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.

[13] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 28, 2013, pp. 1319–1327.

[14] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017, doi: 10.1109/TIP.2017.2662206.

[15] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[16] S. Roth and M. J. Black, "Fields of experts," *Int. J. Comput. Vis.*, vol. 82, no. 2, pp. 205–229, 2009, doi: 10.1007/s11263-008-0197-6.

[17] S. Han et al., "ESE: Efficient speech recognition engine with sparse LSTM on FPGA," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2017, pp. 75–84, doi: 10.1145/3020078.3021745.

[18] S. Cao et al., "Efficient and effective sparse LSTM on FPGA with bank-balanced sparsity," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2019, pp. 63–72, doi: 10.1145/3289602.3293898.

[19] V. Manjunath, R. Yin, D. Lee, A. Ghosh, and P. Panda, "BRIM: Workload-balanced dual-sided bit-serial sparse inference accelerator," arXiv:2607.19431, 2026.

[20] P. Dang, Y. He, and H. Li, "VARA: A voltage-aware ReRAM-based accelerator for energy-efficient computing," arXiv:2609.00421, 2026.

[21] P. Budnik and D. J. Kuck, "The organization and use of parallel memories," *IEEE Trans. Comput.*, vol. C-20, no. 12, pp. 1566–1569, 1971, doi: 10.1109/T-C.1971.223171.